

SUMO 생성기 v3 시나리오 → 학습 공장 투입 결과

현준의 부탁 ①②③ 처리 보고 · 2026-08-18 · 최민서 · GitHub main 커밋 f4df259

한 줄 요약

- 부탁 ① (SUMO 투입 → 트리): 적시경보 규칙 55.3% → 모델 61.7% (같은 오경보 2.04%). 단, SUMO 실차에선 +15p 좋아졌지만 RC 스케일에선 -29p 나빠진 **실차 쏠림** — 그대로 실기 배포 불가.
- 부탁 ② (transformer): 전체 56.6% → 54.6%로 규칙 미달, 트리와 같은 쏠림. ONNX는 만들어 뒀음. 모델 종류보다 데이터 배합(SUMO:RC 위험 시나리오 4:1)이 먼저 고쳐야 할 것.
- 부탁 ③ (GPS 잡음 옵션): 완료. build_dataset_v5.py --gps-noise 로 켜, σ 2.5 m 검증됨.
- 팀 코드 버그 1건 발견 (안 고침, 현준 판단): --streams와 --from-scenario-sim을 같이 주면 학습 단계에서 str/int id 정렬로 죽음. 우회 스크립트 제공.

1. 부탁 ① — SUMO 시나리오를 학습 공장에 투입 (트리)

투입 재료: SUMO 생성기 v3 시나리오 700개(scenario_9001~9700, 진짜 교내 주행 실차 5~25 m/s) × 시나리오당 최근접 2쌍 = 1,401 스트림 + 현준 명령 그대로 --from-scenario-sim 4000 --families --randomize.

학습셋: 3,411,227행 / 5,401 시나리오 (SUMO 1,635,367행·1,401개, scenario_sim 1,775,860행·4,000개). 위험 시나리오: SUMO 1,099 vs scenario_sim 274.

결과 (같은 분할·같은 임계값 0.9018, 평가 1,620 시나리오)

평가 범위	위험 시나리오	적시경보 규칙 → 모델	오경보 규칙 / 모델
전체 (요청하신 숫자)	412	55.3% → 61.7% (+6.4p)	2.04% / 2.04%
SUMO (실차 5~25 m/s)	330	53.9% → 69.1% (+15.2p)	1.92% / 3.50%
scenario_sim (RC 스케일)	82	61.0% → 31.7% (-29.3p)	2.15% / 0.74%

주의 — 전체 숫자만 보면 안 됩니다. +6.4p는 실차 쪽으로 쏠린 평균입니다. SUMO 위험 시나리오가 RC의 4배라 트리가 실차 속도대를 우선 배웠고, RC 스케일(실기 환경)에서는 규칙보다 크게 나빠졌습니다. 이 risk_model_streams.json은 젓슨 실기에 그대로 올리면 안 됩니다.

긍정적인 부분: SUMO만 보면 경보 AI가 실차 속도를 처음 배운 효과가 뚜렷합니다(+15p). 배합만 맞추면 살릴 수 있는 재료입니다.

2. 부탁 ② — transformer (최근 10프레임) 학습

요청 규칙 그대로: 입력은 CSV의 FEATURE_COLUMNS 15열(다른 특징 안 만들), 정답 y_train, 채점은 시나리오 단위 timely_alarm_rate, 규칙과 같은 오경보율(2.17%)에서 임계값. 구조는 risk_transformer_v3와 같은 뼈대(d_model 64.2층·68k 파라미터), 시퀀스는 시나리오 안에서만, 앞 9프레임은 첫 프레임 반복 패딩.

평가 범위	적시경보 규칙 → transformer
전체	56.6% → 54.6% (규칙 미달)
SUMO	54.0% → 60.9%
scenario_sim	65.6% → 32.2%
정지 오경보	0.16% → 0.14%

해석: 트리를 못 이겼지만 첫 시도(6 epoch, 하이퍼파라미터 기본값)라 결론짓기는 이릅니다. 다만 실차 쏠림이 모델 종류와 무관하게 똑같이 나타난 것이 핵심 — 데이터 배합을 먼저 고친 뒤 재학습이 순서입니다.

산출물: models_alarm_tf/alarm_transformer.onnx (입력 [batch,10,15], 출력 logit, ONNX 오차 1.5e-7) + alarm_transformer_scaler.json (mean/scale/threshold/채점 결과 메타). 젓슨 onnxruntime에서 바로 돌아가지만, 위 이유로 지금 배포는 권하지 않습니다.

3. 다음에 해볼 것 (제안 — 현존 판단)

- SUMO 비중 축소: `python sumo_batch_to_streams.py <v5data> --pairs 1 --limit 300 --randomize` → RC:SUMO 위험 시나리오 비율을 1:1 근처로.
- 또는 scenario_sim 표본 가중치 ↑, 또는 속도대(veh_speed 문턱)별로 모델 분리.
- transformer는 배합 고친 뒤 재학습 + epoch/lr 스윙. 스크립트는 --csv/--epochs만 바꾸면 됨.

4. 부탁 ③ — v5 위험지도 데이터에 GPS 잡음 옵션

AI_Model/transformer/build_dataset_local.py에 GPS_NOISE 플래그 추가. 좌표를 읽은 직후 distance_m 계산 전에 `gps_noise.add_gps_noise(df, seed=scenario_seed(...))`를 적용해 거리/TTC/DCPA가 전부 잡음 좌표를 쓰게 함. 기본은 꺼짐(기존 동작 그대로).

```
python build_dataset_v5.py --gps-noise # → training_dataset_v5base_noisy.csv
```

검증(scenario_9001): 깨끗한 거리 대비 오차 평균 2.50 m(σ 2.5 m 실측값과 일치), 최대 11.9 m. 라벨 분포가 상위 등급에서 흩어지는 현장 효과 재현됨 (L3 4→2, L2 60→129).

5. 수정·추가된 파일 (GitHub main f4df259)

모두 rsu/v2x/03_jetson/auto_pipeline/ 아래 (③만 AI_Model/transformer/). 팀 기존 코드는 한 줄도 수정하지 않았습니다 — 전부 신규 파일이거나 제 스크립트 수정입니다.

파일	구분	역할
sumo_batch_to_streams.py	신규	SUMO 시나리오 폴더 일괄 → sim_to_rsu_stream 실행. 시나리오마다 참값 거리 기준 최근접 (차량,보행자) 쌍 K개 자동 선택. 기존 기본값(첫 id)은 hit 행이 0이라 위험 표본이 없었음.
train_streams_model.py	신규	build_dataset_from_streams가 저장한 CSV로 train_and_export만 재실행 (아래 버그 우회). id를 str로 통일 후 팀 함수 그대로 호출.
train_alarm_transformer.py	신규	부탁 ②. 15열-y_train-10프레임-시나리오 단위 적시경보율-규칙과 같은 오경보율-ONNX 내보내기. --csv --out --epochs.
eval_streams_by_source.py	신규	저장된 트리 모델을 원천별(sumo / scenario_sim)로 나눠 적시경보를 채점. 쓸림을 보려면 이걸로.
risk_model_streams.json	산출	부탁 ① 트리 모델 (배포 X — 참고용).
models_alarm_tf/	산출	transformer ONNX + .pt + scaler JSON.
results/*.log	산출	트리·transformer 학습 로그 원문.
결과-SUMO투입_20260818.md	문서	이 PDF와 같은 내용의 마크다운.
AI_Model/transformer/build_dataset_local.py	수정	부탁 ③ GPS_NOISE 옵션 (기본 꺼짐).
AI_Model/transformer/build_dataset_v5.py	수정	--gps-noise 플래그.

미푸시(용량): training_dataset_streams.csv(988 MB), sumo_streams/(1,401쌍). 재생성 약 1.5시간 — 필요하면 말해주면 서버에 올려둘게.

6. 발견한 팀 코드 버그 (안 고쳤음 — 현준이 판단)

build_dataset_from_streams.py --streams ... --from-scenario-sim ... --train을 같이 주면 학습 단계 dataset.split_by_scenario의 sorted(ids)에서 str(sumo id) vs int(scenario_sim id) 비교로 TypeError. 한 원천만 쓰면 안 나서 지금까지 못 만난 버그.

데이터셋 CSV는 그 전에 저장되므로 train_streams_model.py로 학습만 다시 돌리면 됨. 근본 수정은 build_from_streams_dir/build_from_scenario_sim에서 scenario_id를 str로 통일하면 한 줄.

7. 재현 명령

```
cd rsu/v2x/03_jetson/auto_pipeline
python sumo_batch_to_streams.py <v5data> --out sumo_streams --pairs 2 --randomize # (1) ~40 min
python build_dataset_from_streams.py --streams sumo_streams --from-scenario-sim 4000 \
  --families --randomize --out training_dataset_streams.csv # (2) ~40 min, NO --train
python train_streams_model.py --csv training_dataset_streams.csv --model risk_model_streams.json
python eval_streams_by_source.py
python train_alarm_transformer.py --csv training_dataset_streams.csv --out models_alarm_tf --epochs
6 # (3) ~80 min
```

(2)에서 --train을 같이 주면 6장의 버그로 죽으므로 빼고, 학습은 train_streams_model.py로. v5data = AI_Model/transformer/v5data/ (Jetson incoming_data의 scenario_9001~9700 사본, 로컬만).